

Widgets

Są to obiekty w Pythonie reagujące na zdarzenia i umożliwiające obsługę różnych popularnych kontroltek w przeglądarce. Wykorzystane zostaną do utworzenia prostego GUI do obsługi robota. Do ich obsługi konieczne jest zaimportowanie biblioteki ipywidgets.

```
In [1]: import ipywidgets as widgets # zamiast korzystania z pełnej nazwy biblioteki
# ipywidgets wykorzystany zostanie alias o nazwie widgets.
```

```
In [2]: import rospy
from geometry_msgs.msg import Twist
```

```
In [3]: rospy.init_node("ipywidgets_controller")
```

Na zajęciach w których omawiany był Publisher wysyłanie prędkości sterujących odbywało się w następujący sposób:

```
In [4]: # obiekt Publisher
twist_publisher = rospy.Publisher("/turtle1/cmd_vel", Twist, queue_size=10)
```

```
In [5]: # utworzenie i wysłanie wiadomości
some_message = Twist()
some_message.angular.z = 1
some_message.linear.x = 1

twist_publisher.publish(some_message)
```

Można też utworzyć funkcję odpowiedzialną za wysyłanie prędkości postępowej i obrotowej do symulacji dla danego robota na podstawie przekazanych argumentów o wartościach prędkości. Przykładowa funkcja odpowiada za wysyłanie prędkości na topic `/turtle1/cmd_vel`. Funkcja wykorzystuje utworzony wcześniej obiekt Publisher o nazwie `twist_publisher`.

```
In [6]: def move_robot(forward_vel=5, rotation_vel=5):
        '''A function to move turtle from turtlesim simulation

        Args:
            forward_vel (float): Linear velocity
            rotation_vel (float): Angular velocity'''
        message = Twist()
        message.angular.z = rotation_vel
        message.linear.x = forward_vel

        twist_publisher.publish(message)
```

```
In [7]: # wywołanie funkcji z przekazaniem argumentów
move_robot(1, -1)
```

Z punktu widzenia użytkownika konieczne jest dodanie kontroltek umożliwiających wysterowanie prędkości robota bez znajomości programowania.

Slider

Na podstawie funkcji `move_robot` do utworzony zostanie domyślny widget sterujący prędkościami robota. Każda zmiana wartości kontrolki powoduje wywołanie funkcji `move_robot` i wysłanie odczytanych prędkości z kontroltek.

Uwaga

Argumenty funkcji dla, której tworzone będą kontrolki powinny mieć przypisane domyślne wartości. Dla tej funkcji domyślnie zainicjalizowane zostaną dwie kontrolki typu `IntSlider`. Zakres wartości również jest domyślny.

Do utworzenia kontroltek wykorzystywana jest funkcja `interact` z biblioteki `ipywidgets` (używamy aliasu `widgets`). Jako argument przekazywana jest nazwa funkcji (`move_robot`).

```
In [8]: widgets.interact(move_robot)
```

forward_vel  5
rotation_vel  5

```
Out[8]: <function __main__.move_robot(forward_vel=5, rotation_vel=5)>
```

Sterując prędkościami robota, które są w m/s skokowa zmiana o wartości typu `Integer` nie są pożądane. Przydałby się typ `zmiennoprzecinkowy`. W trakcie tworzenia widgetów można określić oddzielnie dla każdego argumentu funkcji typ kontrolki jaki ma

obsługiwać. W tym przypadku zamienimy slidery typu `FloatSlider`, które umożliwiają ustawianie wartości zmiennoprzecinkowych.

Pierwszy argument przy wywołaniu funkcji `widgets.interact` pozostaje bez zmian, jeśli chodzi o nazwę funkcji. Kolejne argumenty jakie podajemy to nazwa argumentu z funkcji i przypisujemy do niej nową kontrolkę.

```
forward_vel=widgets.FloatSlider(min=-10,max=10,value=0)
```

Powyższy zapis oznacza, że dla argumentu **forward_vel** z funkcji **move_robot** utworzony zostanie **FloatSlider** z wartością minimalną -10 i maksymalną 10 oraz wartością początkową 0 pomimo tego, że w funkcji **move_robot** ta wartość wynosi 5.

```
In [9]: widgets.interact(move_robot,
                        forward_vel=widgets.FloatSlider(min=-10,max=10, value=0),
                        rotation_vel=widgets.FloatSlider(min=-3,max=3, value=0))
```

forward_vel  0.00
rotation_vel  0.00

```
Out[9]: <function __main__.move_robot(forward_vel=5, rotation_vel=5)>
```

Wyświetlanie dokumentacji do kontrolki, jako argument

widgets.NazwaKontrolki

```
In [10]: help(widgets.FloatSlider)
```

Help on class `FloatSlider` in module `ipywidgets.widgets.widget_float`:

```
class FloatSlider(BoundedFloat)
|   FloatSlider(*args, **kwargs)
|
|   Slider/trackbar of floating values with the specified range.
|
|   Parameters
|   -----
|   value : float
|       position of the slider
|   min : float
|       minimal position of the slider
|   max : float
|       maximal position of the slider
|   step : float
|       step of the trackbar
|   description : str
|       name of the slider
|   orientation : {horizontal, vertical}
```

Dla różnych argumentów kontrolki mogą być innego typu np. dla **rotation_vel** utworzona została kontrolka pozwalająca użytkownikowi na wprowadzenie wartości. Argument **step** dla kontrolki **FloatSlider** ustawia krok z jakim mają się zmieniać wartości w kontrolce. Kontrolka **FloatText** umożliwia wprowadzanie wartości prędkości z klawiatury.

```
In [11]: widgets.interact(move_robot,
                        forward_vel=widgets.FloatSlider(min=-10,max=10, step=2,value=0),
                        rotation_vel=widgets.FloatText(value=0))
```

forward_vel  0.00
rotation_vel

```
Out[11]: <function __main__.move_robot(forward_vel=5, rotation_vel=5)>
```

Brak podania kontrolki dla argumentu powoduje utworzenie domyślnego slidera.

```
In [12]: widgets.interact(move_robot,
                        rotation_vel=widgets.FloatText(value=0))
```

forward_vel  5
rotation_vel

```
Out[12]: <function __main__.move_robot(forward_vel=5, rotation_vel=5)>
```

Textbox

Teraz czas na wysłanie dowolnej informacji na topicu **/informacja**. Skorzystanie z kontrolki do wprowadzania tekstu spowoduje, że każde wprowadzenie nowego znaku wyśle wiadomość. W terminalu można podejrzeć informację na tym topicu.

```
rostopic echo /informacja
```

Oto przykład:

```
In [13]: from std_msgs.msg import String
def send_msg(wiadomosc):
    pub_info = pub_speed=rospy.Publisher("/informacja",String, queue_size=10)
    msg_info = String()
    msg_info.data = wiadomosc
    pub_info.publish(msg_info)
```

Tym razem przekazywana jest funkcja `send_msg` oraz uzupełniany jej argument `wiadomosc`.

Uwaga

Każda zmiana znaku w wiadomości powoduje wysłanie tekstu, bo wartość w kontrolce zmieniła się.

```
In [14]: widgets.interact(send_msg,
                          wiadomosc=widgets.Text(value='Hello World!'))
```

wiadomosc

```
Out[14]: <function __main__.send_msg(wiadomosc)>
```

Utworzenie kontrolki bez wartości domyślnej dla tekstu. Wartość jest pustym tekstem.

```
In [15]: widgets.interact(send_msg,
                          wiadomosc=widgets.Text())
```

wiadomosc

```
Out[15]: <function __main__.send_msg(wiadomosc)>
```

Odczyt wartości z kontrolki

Do odczytu wartości z kontrolki konieczne jest utworzenie odpowiedniego obiektu, aby w dalszej części programu można było z niego skorzystać i odczytać wartość. Przykład odczytu wartości z kontrolki typu **FloatText**. Utworzony został obiekt **liczba** przechowujący informacje o obiekcie kontrolki typu **FloatText**.

```
In [16]: liczba = widgets.FloatText(
        value=7.5,
        description='Any:',
        disabled=False
    )
odczytana_liczba = liczba.value # odczyt wartości z kontrolki i zapis do zmiennej
print(odczytana_liczba) # wyświetlenie odczytanej wartości
print("typ wartości: ", type(odczytana_liczba)) # wyświetlenie typu odczytanej liczby
```

7.5
typ wartości: <class 'float'>

```
In [17]: liczba
```

Any:

```
In [18]: liczba.value
```

```
Out[18]: 7.5
```

Button

```
In [19]: widgets.ToggleButton(
        value=False, # domyślna wartość dla przycisku
        description='Przykładowy przycisk', # opis przycisku
        disabled=False, # początkowy stan przycisku - wyłączony
        button_style='', # 'success', 'info', 'warning', 'danger' or ''
        tooltip='Description', # Pojawiający się szczegółowy opis po najechaniu na przycisk
        icon='check' # (FontAwesome names without the `fa-` prefix)
    )
```

✓ Przykładowy prz...

W poprzednim przykładzie każdorazowa zmiana tekstu powodowała jego wysłanie. Z punktu widzenia użytkownika konieczne jest wprowadzenie całego tekstu, a następnie wysłanie go do użytkownika.

Teraz czas na modyfikację kodu do wysyłania informacji po wciśnięciu przycisku wyslij. W tym celu utworzona została zmienna `input_text`. Do wyświetlenia kontrolki służy funkcja `display()`, a jako argument przekazywana jest zmienna przechowująca kontrolkę, która ma być wyświetlona. W tym przypadku `input_text`.

Teraz kontrolka z tekstem nie jest w żaden sposób zależna z funkcją od wysyłania tekstu. Odpowiada tylko za przechowywanie informacji tekstowej.

```
In [20]: input_text = widgets.Text(value='Hello World!', disabled=False, description="informacja:")
display(input_text)
```

informacja:

Tekst z kontrolki przechowywanej w zmiennej `input_text` jest odczytywany i zapisywany w zmiennej `odczytany_tekst`. Następnie wyświetlana jest wartość i typ przechowywanej wartości.

```
In [21]: odczytany_tekst = input_text.value
print(odczytany_tekst)
print("typ wartości: ", type(odczytany_tekst))
```

Hello World!
typ wartości: <class 'str'>

Funkcja `send_msg` jako argument przyjmuje informację o stanie przycisku `wyslij` opisanego poniżej (patrz description, zmienna `przycisk_wyslij`), z którego pochodzi żądanie wywołania tej funkcji po każdorazowym kliknięciu przycisku.

Funkcja `send_msg` wykorzystuje wartość tekstu odczytaną ze zmiennej od kontrolki tekstowej `input_text`.

```
In [22]: from std_msgs.msg import String
def send_msg(button_data):
    pub_info = pub_speed=rospy.Publisher("/informacja",String,
                                         queue_size=10)

    msg_info = String()
    msg_info.data = input_text.value
    pub_info.publish(msg_info)
```

Utworzenie przycisku do wysyłania wiadomości.

```
In [23]: przycisk_wyslij = widgets.Button(
    value=False,
    description='wyslij',
    disabled=False,
    button_style='', # 'success', 'info', 'warning', 'danger' or ''
    tooltip='Description',
    icon='check' # (FontAwesome names without the `fa-` prefix)
)
display(przycisk_wyslij)
```

✓ wyslij

W tej chwili po wykonaniu podłączenia na topic'u informacja można zobaczyć, że wiadomość nie jest publikowana.

Przycisk wyslij został utworzony, ale brakuje jeszcze obsługi zdarzenia na kliknięcie. Dla utworzonego obiektu przycisku `przycisk_wyslij` wykorzystywana jest metoda `on_click`, która reaguje na kliknięcie przycisku i powoduje wykonanie funkcji `send_msg` podanej w argumencie. Wywołana funkcja w argumencie posiada informację związane z przyciskiem od którego pochodzi zdarzenie.

```
In [24]: przycisk_wyslij.on_click(send_msg)
```

Dopiero po obsłudze zdarzenia na kliknięcie możliwe jest wysyłanie tekstu.

Checkbox

Przycisk zawierający tylko dwa stany True lub False. Przykład użycia

```
In [25]: checkbox_object = widgets.Checkbox(
    value=False, # Wartość przycisku True - zaznaczony, False - odznaczony
    description='Check me', # Nazwa pola wyboru
    disabled=False,
    indent=False
)
display(checkbox_object)
```

☐ Check me

```
In [26]: checkbox_object.value
```

```
Out[26]: False
```

Dropdown

Lista rozwijana z opcjami do wyboru

```
In [37]: dropdown = widgets.Dropdown(  
    options=['1', '2', '3'], # Opcje do wyboru pojawiające się na liście rozwijanej  
    value='2', # Ustawiona domyślna wartość  
    description='Number:', # Nazwa kontrolki  
    disabled=False,  
)  
display(dropdown)
```

Number:

```
In [30]: # Odczyt wartości  
dropdown.value
```

```
Out[30]: '1'
```

```
In [29]: # Typ wartości  
type(dropdown.value)
```

```
Out[29]: str
```

RadioButtons

Lista przycisków z opcją do wyboru

```
In [62]: radio_button = widgets.RadioButtons(  
    options=['A', 'B', 'C'], # Opcje do wyboru pojawiające się na liście rozwijanej  
    value='B', # Domyślna wartość  
    description="Wersja testu: ", # Opis pola wyboru  
    disabled=False  
)  
display(radio_button)
```

Wersja testu: ☐ A
☒ B
☐ C

```
In [32]: # odczyt wartości z kontroli  
radio_button.value
```

```
Out[32]: 'B'
```

```
In [33]: type(radio_button.value)
```

```
Out[33]: str
```

Wybór wielu opcji

Lista wyboru wielu opcji

```
In [56]: options = widgets.SelectMultiple(
# opcje do wyboru
options=['skanowanie', 'jazda autonomiczna', 'rozpoznawanie piłek tenisowych'],
value=['skanowanie'], # wybrana opcja
rows=7, # Ustawienie liczby wyświetlanych wierszy do wyboru
description='Funkcjonalności robota',
disabled=False
)
display(options)
```

Funkcjonal...

skanowanie
jazda autonomiczna
rozpoznawanie piłek tenisowych

```
In [35]: # odczyt wartości
options.value
```

```
Out[35]: ('skanowanie', 'rozpoznawanie piłek tenisowych')
```

```
In [36]: # Do pola z opisem opcji przypisanie wartości
options2 = widgets.SelectMultiple(
# opcje do wyboru wraz z przypisanymi wartościami
options=[('skanowanie',7), ('jazda autonomiczna', 2),
('rozpoznawanie piłek tenisowych', 3)],
value=[2,3], # zamiast nazw pola podane sa wartości opcji, które zostały wybrane
description= "Funkkcjonalności robota",
disabled=False
)
display(options2)
```

Funkkcjona...

skanowanie
jazda autonomiczna
rozpoznawanie piłek tenisowych

```
In [ ]: options2.value
```

Pobieranie koloru

Kontrolka do wyboru koloru

```
In [41]: color_picker = widgets.ColorPicker(
concise=False,
description='Pick a color',
value='blue', # wartość koloru, domyślnie może być nazwa, ale po wyborze
# jest to wartość podana hexadecymalnie
disabled=False
)
display(color_picker)
```

Pick a color 

```
In [43]: color_picker.value
```

```
Out[43]: '#7e0a0a'
```

Tabs

Do grupowania zakładek i wyświetlania wszystkich w jednej karcie służy **VBox** z omawianej biblioteki. Przyjmuje listę kontroltek do wyświetlenia.

```
In [45]: tab_contents = ['P0', 'P1', 'P2']
# Ustawienie kontroltek w kolejnych zakładkach. Dla zakładek
# tekstowych nadawana jest nazwa taka jak karty. Zmienna children
# zawiera listę z 3 obiektami VBox, w którym każdy z nich zawiera
# 3x IntSlider oraz Text
children = [widgets.VBox([widgets.Text(description=name),
widgets.IntSlider(), widgets.IntSlider(),
widgets.IntSlider())])
```

```

        for name in tab_contents]
tab = widgets.Tab(children = children)

# Ustawienie nazw kolejnych zakładek
for index, page_title in enumerate(tab_contents):
    tab.set_title(index, page_title)
# wyświetlenie zakładek
tab

```

P0	P1	P2
P0 <input type="range"/> 0 <input type="range"/> 0 <input type="range"/> 0		

```

In [46]: # Każdą z kontroltek można zdefiniować oddzielnie, a następnie
# można je dodać do odpowiednich zakładek
kontrolka1 = widgets.IntSlider(description="speed")
kontrolka1_2 = widgets.Checkbox(description="slider_source")
kontrolka2 = widgets.IntSlider(description="cos tam")
kontrolka3 = widgets.Text(description="info")
kontrolka3_2 = widgets.IntSlider(description="forward")
kontrolka3_3 = widgets.IntSlider(description="rotational")

# lista z zakładkami
children = [
    widgets.VBox([kontrolka1, kontrolka1_2]), # pierwsza zakładka,
    # zawiera kontrolka1 i kontrolka1_2
    widgets.VBox([kontrolka2]), # druga zakładka zawiera tylko
    # jedną kontrolkę kontrolka2
    widgets.VBox([kontrolka3, kontrolka3_2, kontrolka3_3])
]

tab = widgets.Tab(children = children)
# ustawianie nazwy zakładek; Kolejne argumenty: id zakładki,
# nazwa zakładki
tab.set_title(0, "P1")
tab.set_title(1, "P2")
tab.set_title(2, "P3")
tab

```

P1	P2	P3
speed <input type="range"/> 0 ☐ slider_source		

Dodatkowe widgety

<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html> (<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html>)

Zmiana stylu i układu kontroltek

Do opisu wyglądu i stylu kontroltek służy obiekt `ipywidgets.Layout`.

```

In [47]: ## Sprawdzanie dostępnych stylów dla kontrolki
b1=widgets.Button(description='To jest przycisk',
                  layout=widgets.Layout(width='50%',
                  height='60px'))

b1.style.keys

```

```

Out[47]: ['_model_module',
'_model_module_version',
'_model_name',
'_view_count',
'_view_module',
'_view_module_version',
'_view_name',
'button_color',
'font_weight']

```

Definiowanie rozmiaru

Na etapie inicjalizacji kontrolki jako argument layout należy przekazać obiekt `ipywidgets.Layout` z podanym rozmiarem kontrolki. W tym przypadku jest to 50% szerokości oraz 60px wysokości. Można też przekazać argumenty dotyczące maksymalnych i minimalnych rozmiarów.

Dostępne parametry:

- `height`,
- `width`,
- `max_height`,
- `max_width`,
- `min_height`,
- `min_width`

```
In [48]: przycisk = widgets.Button(description='To jest przycisk',
                                     layout=widgets.Layout(width='50%', height='60px'))
display(przycisk)
# przycisk zajmuje 50% szerokości obszaru
```

To jest przycisk

Ustawienie siatki do pozycjonowania elementów

```
In [49]: button_layout = widgets.Layout(width='auto', height='auto')
button_style = widgets.ButtonStyle(button_color='darkseagreen')
children = [widgets.Button(layout=button_layout,
                           style=button_style) for i in range(20)]
grid = widgets.GridBox(children=children, # ustawienie kontrolki
                       # które będą kolejno wpisane do siatki
                       layout=widgets.Layout(
                           width='50%',
                           # ustawienie szerokości kolejnych kolumn
                           grid_template_columns='100px 50px 100px 40px',
                           # ustawienie szerokości kolejnych wierszy, rozmiar
                           # dalszych wierszy jest automatyczny
                           grid_template_rows='80px auto 80px 60px',
                           # odstęp w pikselach pomiędzy wierszami,
                           # a później kolumnami
                           grid_gap='5px 10px')
                       )
display(grid)
```

```
In [50]: # wysyłanie różnych wiadomości tekstowych

from std_msgs.msg import String
def send_msg(wiadomosc):
    pub_info = rospy.Publisher("/informacja",String,queue_size=10)
    msg_info = String()
    msg_info.data = wiadomosc
    pub_info.publish(msg_info)
```

```
In [51]: layout = widgets.Layout(width='auto', height='auto')
text1 = widgets.Text(value='Tekst 1', layout=layout)
text2 = widgets.Text(value='Tekst 2', layout=layout)
text3 = widgets.Text(value='Tekst 3', layout=layout)
text4 = widgets.Text(value='Tekst 4', layout=layout)
```



```
text5 = widgets.Text(value='Tekst 5', layout=layout)
```

```
widgets.interact(send_msg, wiadomosc=text1)
widgets.interact(send_msg, wiadomosc=text2)
widgets.interact(send_msg, wiadomosc=text3)
widgets.interact(send_msg, wiadomosc=text4)
widgets.interact(send_msg, wiadomosc=text5)
```

wiadomosc

wiadomosc

wiadomosc

wiadomosc

wiadomosc

Out[51]: <function __main__.send_msg(wiadomosc)>

```
In [52]: children_data=[text1, text2, text3, text4, text5]
grid = widgets.GridBox(children=children_data,
    layout=widgets.Layout(
        width='50%',
        grid_template_columns='150px 150px 150px',
        grid_template_rows='80px auto 80px 60px',
        grid_gap='5px 10px')
    )
display(grid)
```

wiadomosc

wiadomosc

wiadomosc

wiadomosc

wiadomosc

Zmiana kolorów przycisku

```
In [53]: b1 = widgets.Button(description='Custom color')
b1.style.button_color = 'lightgreen'
display(b1)
```

Custom color

Dodatkowe przykłady

<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20Styling.html> (<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20Styling.html>)